

TP2 - Apprentissage automatique

Bastien Jallais

Automne 2025

Contents

1	Introduction	2
2	Montage de l'architecture et entraînement du modèle	2
2.1	Ensemble de données	2
2.2	Traitement de données	2
2.3	Paramètres et Hyperparamètres	3
2.4	Architecture	4
2.4.1	Explication de l'architecture	4
2.4.2	Illustration de l'architecture du modèle	5
2.5	Affichage des résultats d'entraînement	7
2.6	Justification du choix de l'architecture	8
2.7	Récapitulatif	9
3	Évaluation du Modèle	10
3.1	Précision du modèle	10
3.2	Étude de la matrice de confusion	10
3.3	Étude de la matrice d'images mal classées	12
4	Conclusion	13
5	Annexes	13

1 Introduction

Le but de ce travail est de développer une solution capable de faire la distinction à partir d'images de trois espèces marines : la baleine, le requin et le requin-baleine. Les réseaux de neurones convolutifs (CNN), apparus à la suite d'études du cortex visuel du cerveau, sont particulièrement efficaces dans des problèmes de reconnaissance d'images comme le nôtre. Ce sont donc ce type de neurones profonds qu'on utilisera dans ce TP pour répondre à la problématique.

Pour y parvenir, on utilisera la librairie Keras et ensemble de 12 000 images regroupées en trois classes. Le projet sera séparé en deux grandes étapes : la création et l'entraînement du modèle, puis l'évaluation sur des données inédites.

2 Montage de l'architecture et entraînement du modèle

2.1 Ensemble de données

La première étape consiste à séparer les images d'entraînement en deux ensembles : un ensemble destiné à l'apprentissage du modèle (*training set*) et un ensemble utilisé pour évaluer ses performances pendant l'entraînement (*validation set*). Nous adoptons un ratio classique de 80 % pour l'entraînement et 20 % pour la validation.

Contrairement au code original du TP (conçu pour MNIST), il n'est pas nécessaire de spécifier explicitement le nombre total d'images dans le code. En effet, avec la méthode `flow_from_directory()`, Keras analyse automatiquement les dossiers et détermine seul combien d'images sont disponibles pour chaque classe.

Ainsi, le rôle des paramètres `training_batch_size` et `validation_batch_size` ne consiste pas à indiquer le nombre total d'images, mais uniquement la taille des mini-batches, c'est-à-dire le nombre d'images chargées à chaque itération de l'entraînement. Par exemple, un batch size de 32 signifie que le modèle traite 32 images à la fois, jusqu'à parcourir l'ensemble du dossier d'images.

```
1 training_batch_size = 32
2 validation_batch_size = 32
```

Aussi, une petite modification supplémentaire a été apportée au code de `1_Modele.py`, avant de lancer l'entraînement du modèle, on vérifie si un dossier *validation* existe dans `/donnees/`, si oui on ne fait rien, sinon on crée le dossier validation ainsi que trois sous-dossiers à l'intérieur (requin, baleine, requinbaleine) et on remplit ces sous-dossiers à hauteur de 20% des données d'entraînement de chaque catégorie.

2.2 Traitement de données

Après analyse des données, on remarque que toutes les images ont une largeur de 256 pixels. En sachant cela, on peut optimiser le paramètre *image_scale*. Définir celui-ci à 256 peut sembler logique, mais cela serait finalement contreproductif, l'entraînement serait lent dû à la grande quantité de paramètres, et le risque de surapprentissage serait également élevé. Pour les besoins de notre projet, on choisira une taille de 128. Cela permettra un entraînement rapide mais précis, tout en évitant le risque de surapprentissage. Aussi cette taille est donnée comme standard dans les exemples de Keras, preuve de son efficacité.

```
1 image_sclare = 128
```

Une fois ceci fait, il reste maintenant à faire ce qu'on appelle le *data augmentation*. Cela consiste à créer automatiquement des images à partir de celles déjà existantes en leur appliquant des transformations aléatoires. On peut donc, en faisant cela, augmenter artificiellement la diversité du dataset sans ajouter plus de données. Grâce à cela on évite que notre modèle souffre de surapprentissage en raison de données trop similaires, et on augmente donc sa capacité de généralisation. Voici les transformations mises en place :

- **Rotations** : pour changer l'alignement des animaux dans les images.
- **translation** : pour changer la position de l'animal dans l'image .
- **Variation de la luminosité** : important dans ce type de dataset (photo extérieur, paysage, nature).

Voici ce que cela donne dans le code :

```
1 training_data_generator = ImageDataGenerator(  
2     rescale=1./255,  
3     rotation_range=30, # ajout d une rotation  
4     width_shift_range=0.15, # translation horizontal  
5     height_shift_range=0.15, # translation vertical  
6     shear_range=0.15,  
7     zoom_range=0.20,  
8     horizontal_flip=True,  
9     brightness_range=[0.6, 1.4], # On fait varier la luminosite,  
10    fill_mode='nearest'  
11 )
```

2.3 Paramètres et Hyperparamètres

La taille du batch est de 32, cette valeur standard permet de stabiliser le gradient avec un bon compromis entre un gradient trop bruité et un gradient trop lissé. Aussi cette valeur donne les meilleurs résultats au niveau du temps d'entraînement lorsqu'elle est couplée avec le GPU de Colab.

```
1 fit_batch_size = 32 # le nombre d images entrainees ensemble: un batch
```

Le nombre d'époques retenu est de 60, les premiers essais montraient que l'accuracy continuait de s'améliorer jusqu'à 45-50 époques, au-delà, la courbe de validation se stabilise. Pour chercher la meilleure précision possible, on continue l'entraînement tout en rajoutant un *early stop*, comme présenté plus bas.

```
1 fit_epochs = 60 # Le nombre d'epoques
```

Concernant l'optimisateur, Adam a été choisi. Cet optimisateur très utilisé dans les CNN permet une convergence rapide et stable. Un learning rate très faible de (8 e-5) a été choisi car les premiers tests avec des valeurs standard (0,0001 et 0,0005) donnaient un signe de surapprentissage prématuré.

```
1 model.compile(loss='categorical_crossentropy', optimizer=Adam(learning_rate=8e-5), metrics  
    =['accuracy'])
```

Enfin, comme dit précédemment, un early stop a été mis en place avec une patience de 12 pour éviter que le modèle ne s'arrête trop tôt tout en optimisant la précision de celui-ci en lui laissant le temps de s'améliorer correctement.

```
1 earlystop = EarlyStopping(  
2     monitor='val_loss',  
3     patience=12,  
4     restore_best_weights=True  
5 )
```

2.4 Architecture

2.4.1 Explication de l'architecture

Le modèle développé est composé de deux parties : (i) une section d'extraction de caractéristiques (*feature extraction*) reposant sur une succession de couches convolutionnelles, et (ii) une section de classification entièrement connectée (*fully connected*). L'architecture complète est détaillée ci-dessous.

Feature Extraction

La partie convolutionnelle du réseau comprend cinq blocs chacun constitué d'une couche de convolution suivie d'une normalisation de lot, d'une fonction d'activation ReLU et d'un échantillonnage par max-pooling. Les blocs sont les suivants :

- **Bloc 1 :**
 - Convolution 2D : 32 filtres, noyau 3×3 , *padding* = **same**
 - Batch Normalization
 - Activation ReLU
 - Max Pooling : taille 2×2
- **Bloc 2 :**
 - Convolution 2D : 64 filtres, noyau 3×3 , *padding* = **same**
 - Batch Normalization
 - Activation ReLU
 - Max Pooling : 2×2
- **Bloc 3 :**
 - Convolution 2D : 128 filtres, noyau 3×3
 - Batch Normalization
 - Activation ReLU
 - Max Pooling : 2×2
- **Bloc 4 :**
 - Convolution 2D : 256 filtres, noyau 3×3
 - Batch Normalization
 - Activation ReLU
 - Max Pooling : 2×2
- **Bloc 5 :**
 - Convolution 2D : 256 filtres, noyau 3×3 , activation ReLU
 - Max Pooling : 2×2

Fully Connected

La partie entièrement connectée du modèle est constituée des couches suivantes :

- Couche **Flatten** pour convertir les cartes de caractéristiques en vecteur.
- Couche dense de 256 neurones avec activation ReLU.
- **Dropout 0.5** pour réduire le surapprentissage.
- Couche dense de 128 neurones avec activation ReLU.
- **Dropout 0.4**.
- Couche dense finale de 3 neurones suivie d'une activation **Softmax** permettant la classification multiclasse (*baleine*, *requin*, *requin-baleine*).

Fonctions d'activation et Dropout

L'architecture utilise la fonction d'activation ReLU dans toutes les couches internes, car elle facilite la convergence et réduit le risque de gradient nul. La couche de sortie utilise une activation Softmax adaptée à la classification à trois classes. Du dropout est appliqué dans les couches denses finales afin de limiter le surapprentissage, avec des taux de 0.5 et 0.4 respectivement.

2.4.2 Illustration de l'architecture du modèle

Pour des raisons de lisibilité, les schémas présentés ci-dessous ont été générés à partir d'une version simplifiée du réseau à l'aide de l'outil NN-SVG. Les dimensions exactes des cartes de caractéristiques et la taille réelle du vecteur flatten (4096 neurones) ont été compressées afin de ne pas dépasser un maximum d'environ 16 neurones ou filtres par couche.

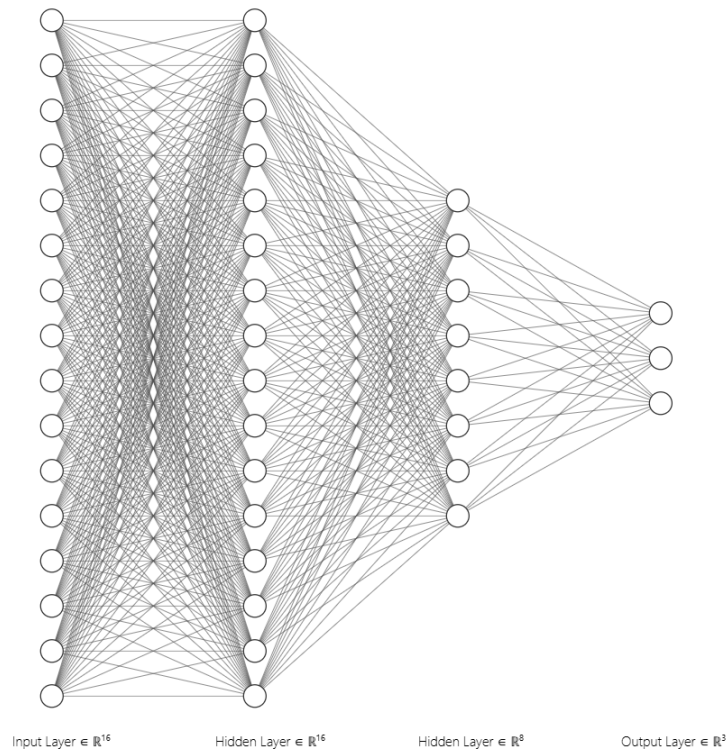


Figure 1: Représentation simplifiée de la partie entièrement connectée (FCNN) du modèle.

Figure 1 illustre la partie entièrement connectée du modèle. On y retrouve la structure du vecteur flatten (compressé pour la lisibilité) suivie de deux couches denses accompagnées de dropout, puis la sortie Softmax à trois neurones. Ce schéma met en évidence la phase finale de décision du réseau.



Figure 2: Vue d'ensemble du modèle complet tel que généré par `visualkeras`.

Légende des couleurs de l'architecture (VisualKeras)

La Figure 2 utilise un code couleur permettant de distinguer rapidement les différents types de couches du modèle :

- ■ **InputLayer** : entrée du réseau, format $(128 \times 128 \times 3)$.

- **Conv2D** : couches convolutionnelles extrayant les motifs locaux.
- **BatchNormalization** : normalisation des activations pour stabiliser l'apprentissage.
- **Activation (ReLU)** : fonction d'activation introduisant la non-linéarité.
- **MaxPooling2D** : réduction de la résolution spatiale.
- **Flatten** : conversion de la carte de caractéristiques en vecteur.
- **Dense** : couches entièrement connectées assurant la classification finale.
- **Dropout** : régularisation pour limiter le surapprentissage.
- **Softmax** : couche de sortie donnant les probabilités pour les 3 classes.

Figure 2 fournit une vue d'ensemble du modèle complet telle que générée automatiquement par VisualKeras. Ce schéma montre la succession des couches convolutionnelles, des normalisations, du pooling et des couches fully-connected, offrant une vision claire du flux de données à travers tout le réseau.

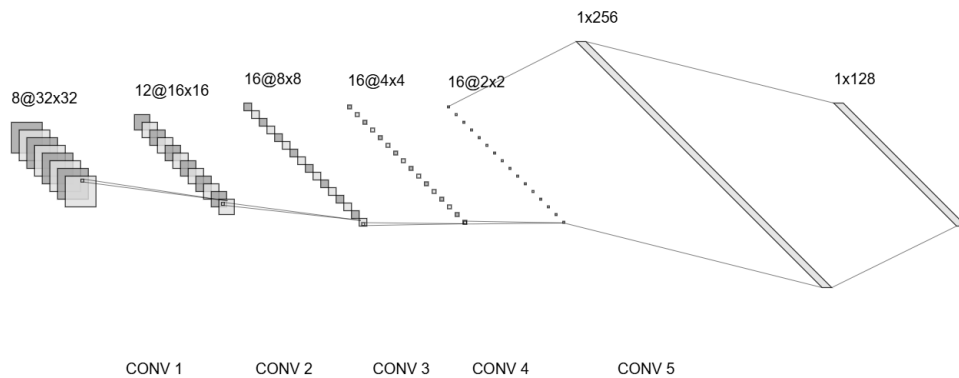


Figure 3: Schéma convolutionnel (style LeNet) illustrant la succession des cinq blocs CNN.

Figure 3 représente une version simplifiée (style LeNet) de la partie convolutionnelle du modèle. Les dimensions ont été compressées pour illustrer visuellement les cinq blocs convolutionnels successifs et la réduction progressive de la résolution spatiale tout en augmentant la profondeur.

2.5 Affichage des résultats d'entraînement

Dans un premier temps on regarde le temps total de l'entraînement du modèle en minutes. Pour cela on lance le timer juste avant `model.fit` et on arrête juste après. On fait cela pour ne pas prendre en compte le chargement, la compilation, ou la mise en place du GPU.

```
1 # Debut du chrono juste avant l'entraînement du modele
2 start = time.time()
3
4 # entraînement du modele
5 # Ici on utilise le generator complet, afin d'utiliser correctement le data augmentation
6
7 classfier = model.fit(
8     training_generator,
9     epochs=fit_epochs,
10    validation_data=validation_generator,
11    callbacks=[modelcheckpoint],
12    verbose=1
13 )
14
15 # Fin du chrono juste apres l'entraînement du modele
16 end = time.time()
```

Le temps total d'entraînement du modèle est de 53.04 minutes. Le modèle atteint une erreur minimale (*Minimum Loss*) de 0.1840 sur les données d'entraînement, ainsi qu'une exactitude maximale (*Maximum Training Accuracy*) de 93.11%. L'exactitude maximale obtenue sur les données de validation est de 90.87%.

Les figures suivantes présentent l'évolution de l'exactitude ainsi que de la perte au fil des époques pour l'entraînement et la validation.

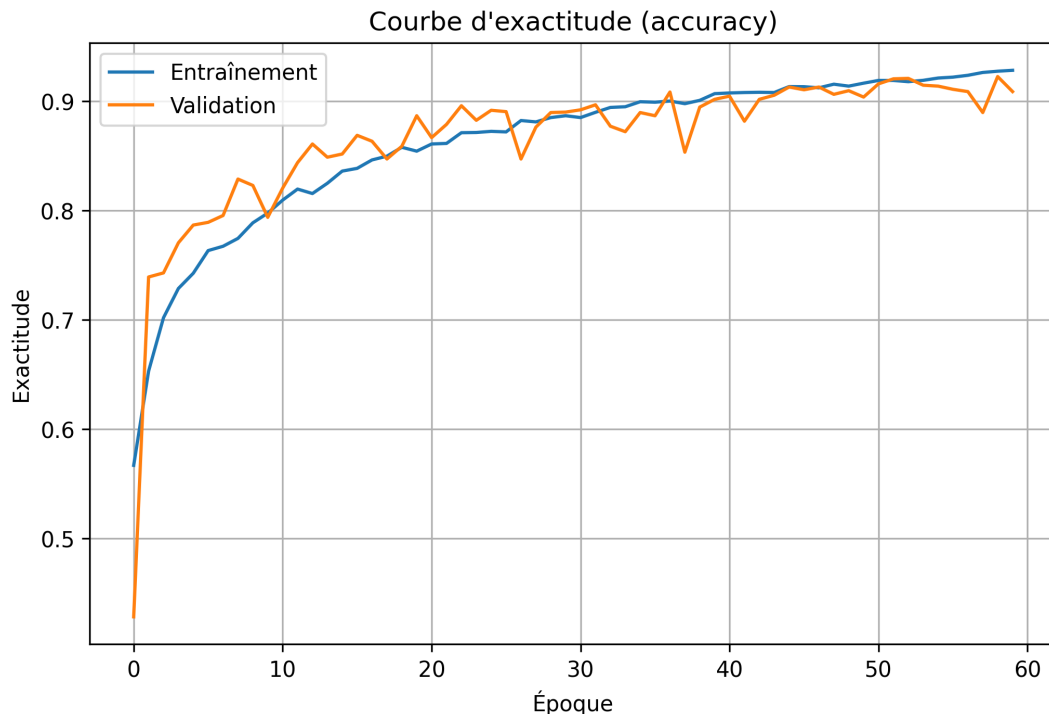


Figure 4: Courbe d'exactitude (accuracy) de l'entraînement et de la validation au fil des époques.

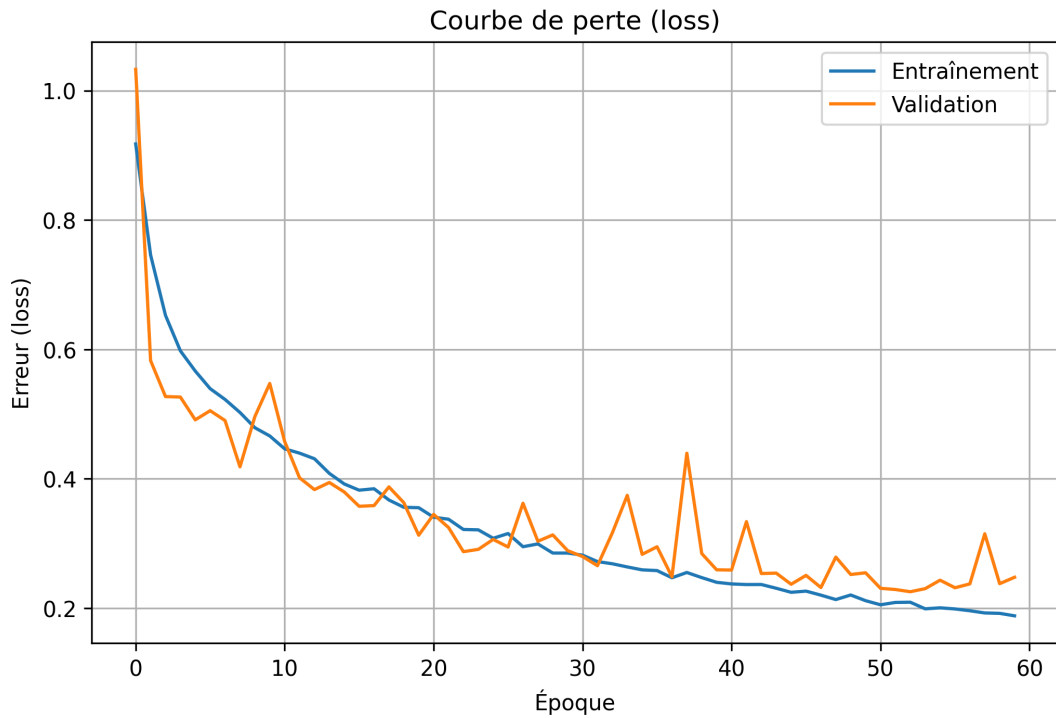


Figure 5: Courbe de perte (loss) de l'entraînement et de la validation au fil des époques.

Analyse des courbes

La courbe d'exactitude montre une progression régulière jusqu'à atteindre environ 93% pour l'entraînement et 91% pour la validation. Les deux courbes restent proches, ce qui indique une bonne capacité de généralisation du modèle.

2.6 Justification du choix de l'architecture

Profondeur du réseau. L'utilisation de plusieurs blocs convolutionnels avec un nombre de filtres croissant (32, 64, 128, 256) permet d'extraire des caractéristiques de plus en plus complexes. Les premiers blocs capturent des motifs simples (bords, textures), tandis que les blocs plus profonds apprennent des structures plus abstraites.

Normalisation de lot (Batch Normalization). Chaque couche convolutionnelle est suivie d'une normalisation de lot, ce qui stabilise la distribution des activations et accélère la convergence. Ce mécanisme permet d'entraîner le modèle avec un taux d'apprentissage relativement faible tout en maintenant une dynamique d'entraînement stable. Il contribue également à réduire le surapprentissage.

Fonctions d'activation ReLU. L'activation ReLU a été retenue pour l'ensemble des couches internes, car elle évite le problème du gradient évanescent et permet un entraînement plus rapide. ReLU favorise une meilleure propagation du gradient dans les couches profondes.

Structure entièrement connectée et Dropout. Les deux couches denses de 256 et 128 neurones permettent de combiner les caractéristiques extraites dans la partie convolutionnelle. L'utilisation du dropout (0.5 puis 0.4) a été déterminante pour éviter le surapprentissage, particulièrement visible lors des premiers. Ce mécanisme force le réseau à ne pas dépendre excessivement de certains neurones, améliorant ainsi sa capacité de généralisation.

2.7 Récapitulatif

Section	Résumé des choix et paramètres
2.1 Ensemble de données	• Entraînement : 4 000 images par classe (12 000 total) • Validation : 20% de l'entraînement, soit 800 images par classe (2 400 total) • Test : 1 000 images par classe (3 000 total) • Couleur : RGB, taille des images : 128×128
2.2 Traitement des données	• Normalisation : <code>rescale=1./255</code> • Data Augmentation : – Rotation jusqu'à 30° – Décalage horizontal/vertical : 0.15 – Zoom : 0.20 – Shear : 0.15 – Flip horizontal – Variation de luminosité : $[0.6, 1.4]$
2.3 Paramètres et Hyperparamètres	• Optimiseur : Adam, taux d'apprentissage : 8×10^{-5} • Batch size : 32 • Nombre d'époques : 60 • Early Stopping
2.4 Résultats d'entraînement	• Temps total d'entraînement : 53.04 minutes • Minimum Loss (entraînement) : 0.1840 • Maximum Accuracy (entraînement) : 93.11% • Maximum Accuracy (validation) : 90.87% • Figures d'exactitude et de perte incluses dans le rapport

Table 1: Résumé des paramètres, de l'architecture et des résultats d'entraînement du modèle CNN.

3 Évaluation du Modèle

3.1 Précision du modèle

Une fois l'entraînement terminé, on passe à l'évaluation de notre modèle sur les données de test. On remarque que celui-ci obtient une accuracy d'environ 92 % ainsi qu'une perte de 0,23, ce qui indique une bonne capacité du modèle à classer efficacement les différentes classes en dépassant la limite de minimum 90 % imposée par les consignes du TP :

```
1 > Test loss (Erreur): 0.2347814291715622
2 > Test precision: 0.918666660785675
```

3.2 Étude de la matrice de confusion

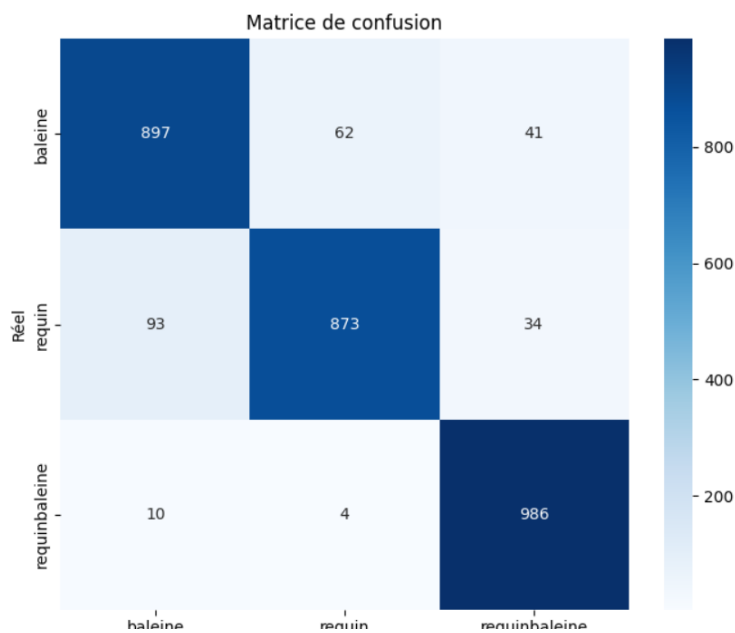


Figure 6: Matrice de confusion du modèle sur l'ensemble de test.

Mesures de performance par classe

On rappelle les formules des métriques utilisées :

- **Précision (Precision) :**

$$P = \frac{TP}{TP + FP}$$

- **Rappel (Recall) :**

$$R = \frac{TP}{TP + FN}$$

- **F1-score :**

$$F1 = \frac{2PR}{P + R}$$

Classe 0 — Baleine

$$P_{\text{Baleine}} = \frac{897}{897 + 103} = 0.897$$

$$R_{\text{Baleine}} = \frac{897}{897 + 103} = 0.897$$

$$F1_{\text{Baleine}} = \frac{2 \times 0.897 \times 0.897}{0.897 + 0.897} = 0.897$$

Classe 1 — Requin

$$P_{\text{Requin}} = \frac{873}{873 + 66} = \frac{873}{939} \approx 0.931$$

$$R_{\text{Requin}} = \frac{873}{873 + 127} = \frac{873}{1000} = 0.873$$

$$F1_{\text{Requin}} = \frac{2 \times 0.931 \times 0.873}{0.931 + 0.873} \approx 0.901$$

Classe 2 — Requin-baleine

$$P_{\text{RequinBaleine}} = \frac{986}{986 + 75} = \frac{986}{1061} \approx 0.929$$

$$R_{\text{RequinBaleine}} = \frac{986}{986 + 14} = \frac{986}{1000} = 0.986$$

$$F1_{\text{RequinBaleine}} = \frac{2 \times 0.929 \times 0.986}{0.929 + 0.986} \approx 0.958$$

On remarque que la classe Requin-baleine est la mieux maîtrisée par le modèle ($F1 = 0,958$). La classe requin est quant à elle la plus compliquée avec un taux de faux positifs de 127, ce qui réduit le rappel. La classe Baleine est équilibrée avec un score $F1 = 0,897$.

3.3 Étude de la matrice d'images mal classées

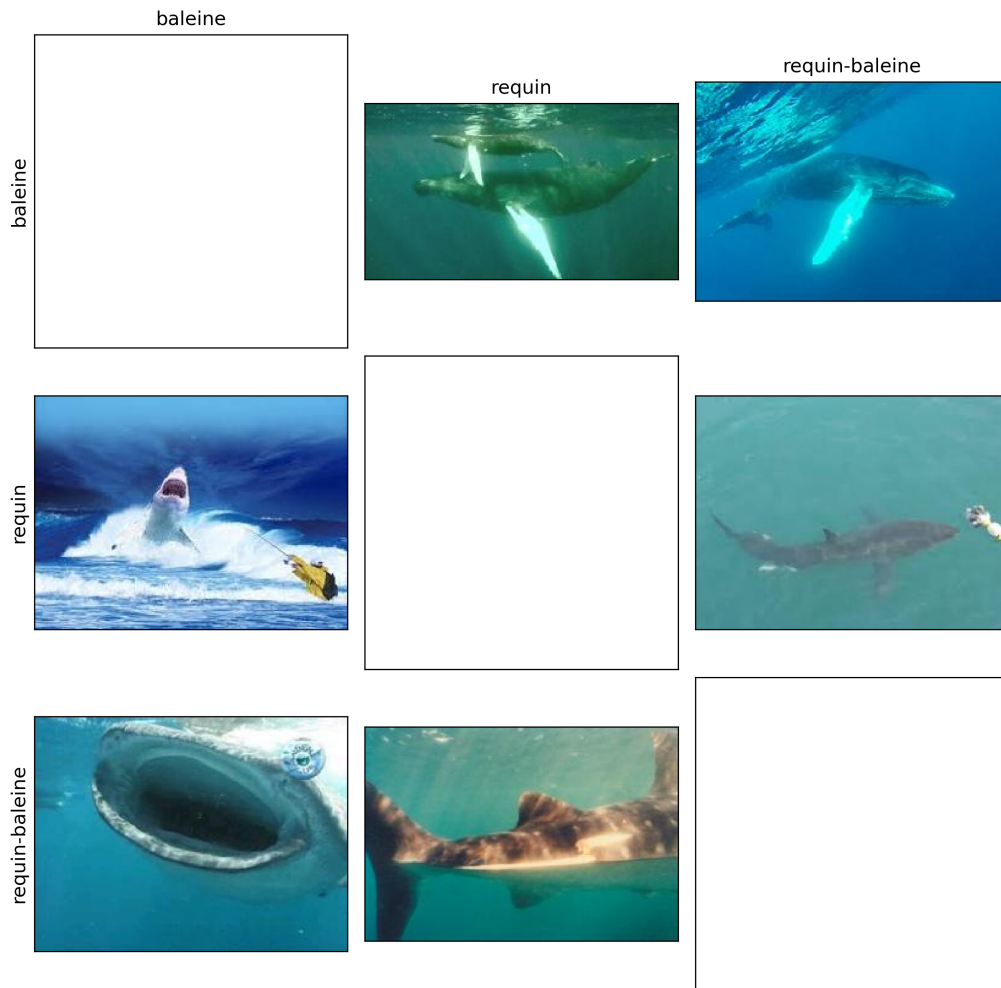


Figure 7: Matrice des images mal classées (réel $\rightarrow$ prédit).

Étude de la matrice d'images mal classées

La matrice d'images mal classées (Figure 7) permet de visualiser les erreurs du modèle pour chaque combinaison (*réel* $\rightarrow$ *prédit*). Les confusions proviennent principalement de situations où l'animal est partiellement visible, l'angle de prise de vue masque les traits distinctifs, ou encore lorsque la luminosité et la couleur de l'eau réduisent le contraste.

Les baleines sont parfois confondues avec les requins ou les requins-baleines lorsque seule la silhouette est visible. Les requins peuvent être classés à tort lorsqu'ils apparaissent de loin ou sous un angle atypique. Enfin, les rares erreurs sur les requins-baleines surviennent lorsque leurs motifs caractéristiques ne sont pas visibles. Dans l'ensemble, les erreurs sont cohérentes et s'expliquent par des conditions visuelles défavorables plutôt qu'un défaut important du modèle.

4 Conclusion

Ce travail a permis de développer et d'évaluer un réseau de neurones convolutif capable de distinguer trois espèces marines à partir d'images. Plusieurs difficultés ont été rencontrées au cours du projet. La première concernait la recherche d'une architecture suffisamment bonne pour atteindre une précision minimale de 90%, sans pour autant conduire à du surapprentissage.

Un autre obstacle a été la difficulté à dépasser le palier de 85–88 % d'accuracy sur les données de test. Ce plafond a finalement été surmonté grâce à une combinaison d'augmentations de données plus puissantes, d'une augmentation du nombre d'époques, de l'ajout d'un mécanisme d'*early stopping* et d'un réglage du learning rate sur l'optimiseur Adam. Ces ajustements ont permis d'améliorer la généralisation du modèle et d'atteindre une précision supérieure à 90 %.

Plusieurs pistes d'amélioration pourraient encore être envisagées. Une exploration plus exhaustive des techniques de régularisation (dropout plus élevé, L2), l'essai d'architectures préentraînées (*transfer learning*) ou l'utilisation de méthodes de réduction de dimension avant la phase dense pourraient offrir de meilleures performances tout en réduisant le nombre de paramètres. L'intégration d'un schéma de recherche automatique d'hyperparamètres pourrait également optimiser davantage le modèle.

De manière générale, ce projet a mis en lumière l'importance du choix d'architecture, de la préparation des données et du réglage des hyperparamètres dans la conception d'un CNN performant pour la classification d'images.

5 Annexes

Outils utilisés pour les figures décrivant l'architecture du modèle :

- VisualKeras
- <https://alexlenail.me/NN-SVG/index.html>
- <https://alexlenail.me/NN-SVG/LeNet.html>